

Recursividad

Describe el concepto de recursividad

- Técnica de programación
- Función que se llama así misma
- Proceso en el cual se resuelve un problema dividiéndolo en sub-problemas hasta llegar a un caso base
- Resolver un problema complejo dividiéndolo en problemas más simples

"Técnica de programación que se basa en una función llamarse a si misma para resolver un problema complejo dividiéndolo en problemas más simples."

Ejemplos de problemas recursivos

- El factorial de un número
- Serie de fibonacci

¿Qué significan los niveles de recursividad?

- Profundidad de llamas recursivas
- Nuevo nivel de pila de llamadas
- El caso base es el nivel final, el problema más simple
- Está limitado por la memoria, pero los lenguajes de programación limitan la cantidad de niveles,

"Es la cantidad de veces que una función que se llama así misma, creando un nuevo nivel en la pila de llamadas que no será liberada hasta el retorno o finalización de la función. El caso base es el nivel final o el problema más simple de resolver."

Tipos de recursividad

- Recursividad simple: La función solo se llama una vez a si misma, una única vez por nivel.
- Recursividad anidada: Un nivel de recursión se pasa como argumento a otra llamada del mismo nivel.
- Recursividad indirecta: La función A llama a la función B, y esta a su vez vuelve a llamar a la función A, creando niveles alternados entre distintas funciones (No muy usada).
- Recursividad Múltiple: Ocurre cuando dentro del cuerpo de una función se realizan dos o más llamadas a la misma función. Esto provoca que el flujo de ejecución no sea una línea simple, sino que se ramifique como un árbol.

Partes de una función recursiva

- Parámetros de estado: Las funciones recursivas requieren de parametros para una mayor utilidad. Los parámetros cambian en cada llamada o en cada nivel de recursión.
- Caso base (Condición de parada): Para evitar el *Stack Overflow* (Agotar la memoria asignada de la pila) se necesita saber el caso más pequeño en el cual la función tiene que detenerse.

- Paso recursivo (Llamada de retorno): Es necesario modificar los parametros (O los argumentos al usar la función) combinando el resultado actual con el de la siguiente llamada. Cambia el estado de los parametros para llegar al caso base.

Ventajas y desventajas de la recursividad

Ventajas

1. Elegante: Las soluciones recursivas suelen verse más legibles (Para quienes sepan de la recursividad) y bastante corto de escribir.
2. Estrategia de divide y vencerás: Diversos algoritmos hacen uso por *dividir un problema complejo en sus partes más pequeñas* que es lo que hace una función recursiva en cada nivel de recursión.
3. Potencia: Varios problemas tienen naturaleza recursiva y a simple vista para ellos

Desventajas

1. Memoria: Hace mayor uso de memoria a diferencia de un simple bucle. Una función recursiva es capaz de sobrecargar la memoria.
2. Velocidad: Una función recursiva siempre será más lenta que una función iterativa